Smooth Drone Formation Flying: Fast Formation Configuration (Final Report)

Frederik Markwell
Computer Science and Software Engineering
University of Canterbury
Christchurch, New Zealand
fma107@uclive.ac.nz

***Abstract*—Drones flying together in a coordinated swarm have a number of uses. A key difficulty encountered in work with these swarms is managing the status of each drone in the swarm - it is difficult to manually configure each drone. This report presents a draft solution for transferring software to the drones. The solution improves upon manual practices but has areas for improvement. The project has been managed by creating a list of user stories based on gathered requirements, then prioritising them and working on them in sequence.**

I. Introduction / Background

Drones, also known as unmanned aerial vehicles, have seen increasing use recent. Small quad-copters have become a feasible solution to a number of real world issues [1]. The majority of these applications at present use only a single drone.

A *drone swarm* is when several drones act together in a controlled formation. This has greater flexibility than a single drone, but is much more complex for a human operator to control.

One major advantage of a formation of drones is that because of their separation, signals from the ground will reach them at different times. This allows for the swarm to triangulate the position of the signal much more accurately than a single drone.

Possible applications of this technology include search and rescue, tracking of flying insects and firefighting [2], [3].

This project explores various methods to improve performance of flight tests of a swarm of drones. Flight tests in previous iterations of the project have encountered difficulties due to the complexity of configuring a number of drones in challenging conditions (e.g. away from internet and power).

The drones used for this project have two main parts. Each drone has a Raspberry Pi mounted on its underside. This small computer runs Ubuntu, and is used for the more complex code that coordinates positioning information between different members of the swam. It uses the Robot Operating System (ROS) software framework to communicate between different processes in the control system. When required, the Pi makes a request to the drones flight controller to change to a specific position or velocity. The flight controller manages the individual drone, adjusting motor power to follow the instructions of the Pi. An example of the drone used in this project can be seen in Figure 1.

In section II, we describe the objectives of the project as a whole and this project more specifically. In section III, we give a general background and outline the existing solutions to the problem. In section IV we describe the proposed solution. This includes details of the technology stack and design. The same section also includes a discussion of how work on the project is organised. Finally, section V discusses the results.

Fig. 1. A drone used in this project takes flight automatically.

II. Objectives

The project is supervised by Prof. Andreas Willig and Dr Graeme Woodward. It is a team project, consisting of two software engineering students, two mechatronics engineering students, and one electrical engineering student.

This project continues the work of a previous project to control a swarm of roughly five drones. The previous work on the project resulted in a formation flight of three drones around a square track. Limitations in path planning and technical issues with the drones prevented tests of a larger formation. Ultimately, the goal of this project is to address the limitations of this system and conduct a successful flight test with the entire drone swarm.

The project is divided into five sub-projects, each with a different objective.

1) *Data Dissemination Protocol*
   Implement a protocol to share real-time information across the network of drones.
2) *Path Planning and Trajectory Following*
   Calculate the trajectory of individual drones based on

the path of the entire swarm, and implement relevant trajectory following algorithms to make these movements.

3) *Drone Life Cycle and Safety Management*
Create a framework for managing the state of the drone, allowing for complete user customisation of the states and transitions between them.
4) *Fast Formation Configuration and Diagnosis*
Implement a framework and user interface allowing easy access to the drones' configuration and real-time data.

This report focuses on sub-project four. Technical difficulties with drone configuration and monitoring were reported by the previous group to have seriously hindered flight test attempts. We aim to solve this by creating a simple user interface that will allow users to deploy software easily and consistently to the drones.

III. Existing Solutions

Traditionally, software updates for embedded devices are done by either physically replacing storage modules, or by connecting to another device with a wire [4]. This method is slow and often inconvenient, as it means you have to return the devices to a physical location to transfer the software over.

The process developed by previous teams working on this project for transferring software to the drones used this method. It involved connecting a drone to a mouse, keyboard, and screen, then running `git clone` to download the source code. Code that needs to be compiled is compiled directly on the drone.

This software can then be transferred to other drones by removing the micro-SD card from the raspberry Pi and cloning it several times. Since the drones now have identical software, the final step is to change configuration files on each drone so they can perform their own unique behaviour.

This process is quite slow and means that changes to the software cannot be easily performed in the field. There is also risk of human error when the configuration files are changed, which makes for confusing debugging.

Wireless transfer technology is emerging as a solution to this problem. This method is particularly popular in the automotive industry, where updating a vehicle's software manually would require a trip to a licensed workshop [5].

Varadharajan et al. [6] present a method for updating code for a swarm of robotic devices over the air (OTA). They use a custom OTA protocol to relay changes to software across a network of devices. Consistency is ensured using a gossip based algorithm that allows the nodes to reach a consensus on the correct version.

This method was successful, decreasing the time to update software on a swarm of four UAVs to around 100 seconds, whereas previously it would have taken 10 minutes. However, it is a complex system requiring implementation of several new protocols, which means the scope was too great for a project of this size.

Alternate deployment systems are popular outside of the embedded systems space. Docker is a platform that allows software and its dependencies to be packaged and easily distributed together [7]. It is primarily used for continuous integration and deployment. Docker provides similar isolation benefits to using a virtual machine, but without as much performance overhead because it shares the kernel with the host system [8] It is also portable, meaning that images created on one machine can be easily moved and used on another [9].

IV. Proposed Solution

A. Design and Implementation

Figure 2 shows a graphical representation of entire system architecture.

1) Drone Detection: A key difficulty with connecting to the drones is that their IP addresses are dynamically assigned using DHCP [1] when they connect to the network. In the past, this was not a problem, as you could simply manually look at the devices connected to the access point and type in the IP address to connect to it. However, this automated solution needs a way of automatically discovering new drones that have connected to the network.

Several possible solutions were available. Firstly, we could just keep the static configuration. The MAC addresses of the drones will not change, so we could configure our transfer process to only send out the software to devices that we have on file. This option was rejected because it was not flexible enough. The long term goal of the project is to scale up to larger formations, and inputting the MAC address of every new drone you add would slow the process down. Additionally, "hard-coding" the MAC addresses would make swapping out hardware components more complicated.

An alternative approach is to use the device running the management interface as your access point. This allows you to control the DHCP server, and thus you know what addresses you are assigning to various devices. We rejected this option because it also reduces the flexibility of the system. All devices would have to connect directly to the management interface, which precludes the possibility of using a more advanced or specialised device as a wireless access point.

The final method that we decided on was to detect the drones by examining the devices that are connected to the wireless access point. We send out a packet to the broadcast MAC address, which all devices connected to the network will reply to. We cannot assume that all devices on the network are drones, as other devices such as laptops may also connect. To solve this, we send an HTTP request to a well specified port and endpoint. If we get a successful reply, we can assume that it is a drone. These requests are done in parallel to speed up the time we would otherwise spend waiting for all devices on the network to either reply or not reply.

2) Continuous Integration / Deployment: The code for the software is tracked by Git, and stored on the University of Canterbury's own GitLab instance. [2]

The automatic software building is managed by a GitLab runner, which is installed on a virtual machine inside the

[1] See https://en.wikipedia.org/wiki/Dynamic_Host_Configuration_Protocol
[2] See https://eng-git.canterbury.ac.nz

university network. The runner detects when changes are pushed to the GitLab server, and automatically obtains a copy of the new code. It then runs a predefined script to build the software.

The software solution produced by this project depends on a number of external libraries, mostly used for interfacing with networking hardware and communicating with the flight controller. It is important that these dependencies be installed on the drones in a manner that is reproducible.

For this reason, we chose to containerise the system using Docker. The GitLab runner calls Docker to build a new image, which then uses the "dockerfile" included with the source code to build the application. This is processed in layers, with caches of each layer. This means that the entire container does not have to be rebuilt from scratch if code changes, only the later stages will need to change. This makes the build process relatively efficient.

Using a virtual machine instead of containerisation was considered, but rejected because of potential performance considerations. Docker allows for a much more lightweight solution because it uses more of the underlying operating system.

The initial approach for transferring the Docker image to the Raspberry Pi was to simply compress the image and then transfer it a normal file sharing tool such as `scp` or `rsync`. However, the image of the current version of the software was approximately 2 gigabytes in size, which could make for a slow file transfer, especially on non-ideal internet connections.

Instead, we host a Docker registry on the virtual machine. This allows us to take advantage of Docker's caching abilities. When you make a `docker pull`, it analyses the layers of the software currently present and compares them with those on the remote. This means that if changes are made to only the source code, only those changes will need to be sent over the network. The majority of the file size of the image is the large dependencies, which will change infrequently.

The Docker registry is made accessible to the internet so that the software can be downloaded from anywhere. We encrypt the connection using TLS and a certificate from Let's Encrypt [3] so that we can be sure that our software is not being tampered with while it is downloading.

3) Field Deployment: It is possible that the drones may be used in areas where access to the internet is not readily available. This means that instead of transferring the software directly to the drones from the virtual machine, we will have to first transfer it to a management device, such as a laptop, and then from there transfer it to the drones.

Before going out into the field, the device running the management interface can be told to contact the server hosted by the virtual machine, and it will run a `docker pull` command to download, over the internet, any layers of the image that have changed since the last version. This image is stored locally on the machine. This can be repeated several times if multiple versions of the software are being tested.

[3]See https://letsencrypt.org/

Based on consultation with team members later in the year, an option was added to allow users to build a docker image directly from the code on their machine. This allows you to make last minute changes to the software in the field if needed.

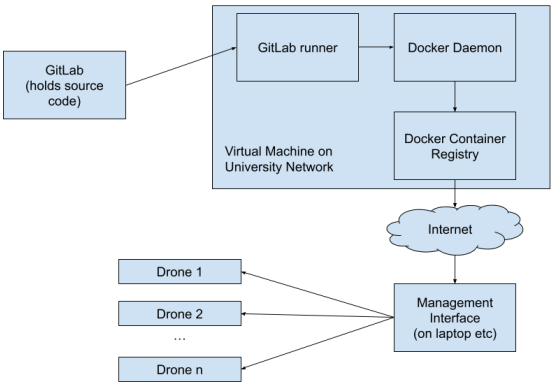

Fig. 2. A graphical representation of the how software is automatically built and deployed on the drones.

The process for transferring software from the managing device to the drones is more complex. As before, we host a Docker registry so that we can take advantage of Docker's caching features. However, as we are now transferring images across only the local network, we cannot obtain a TLS certificate from Let's Encrypt as we did before. [4].

Instead, we create a self signed certificate, and encrypt the connection to the Docker registry with that certificate. We do this whenever the management interface starts up, as the certificate has to change based on the management device's current IP address.

We also have to transfer the certificate to each drone, so that it can verify the connection is valid. We do this using `scp`, and pre-generated keys which allow us to not have to enter a password.

Finally, to ensure correct verification of the security certificate, we update the system time on every drone to match the time on the management device. Raspberry Pis lack a real time clock, which means they "lose time" when not powered. Having the clock set correctly is also useful for cross-referencing logging timestamps.

Each drone can then `docker pull` the layers of the image that are different between its version and the version on that computer. This is a quick process as the image is only transferred across the local network, not across the internet.

Figure 3 shows the transfer of software in the field over time.

[4]A certificate issued by a certificate authority for local a IP addresses is impossible, as many different people use that IP address in their own local network

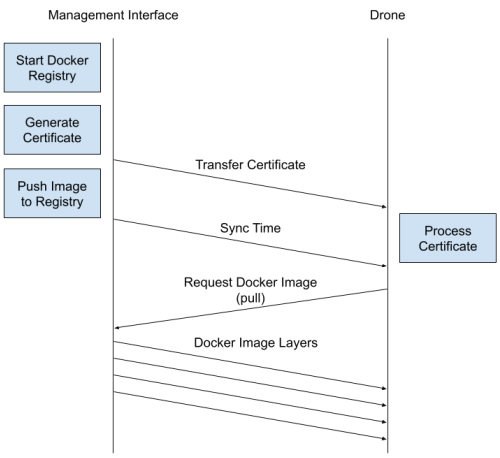

Fig. 3. The transfer process from the management device to one drone with respect to time.

4) Management Interface: The management interface software is composed of two main parts. The first part is the back-end of the management interface. It has been designed as a Python web server using Flask [5] so that the front-end interface can be easily switched out with something different if required. Python was chosen as it has good support for interacting with sub-processes. [6] It is also a language that future students working on the project are likely to be familiar with, speeding knowledge transfer.

The final part is a front-end graphical interface for the management system. This will interact with the back-end system to display the information to the end user. It uses the front-end JavaScript library React, with TypeScript. We chose to use a JavaScript framework in conjunction with a REST API because it allows for very flexible and fast user interface development. If a more traditional desktop application is required in the future, it can be packaged using tools like electron.

B. Method and Project Management

Generally, the different parts of the project are managed individually. We work on our own sub-part of the project following our own time management. In weekly meetings, we discuss progress made and future plans. This helps to coordinate the parts of the solution where our sub-projects will need to interact.

At the beginning of this project, the requirements for the software solution were somewhat vague. Therefore, some requirements gathering was conducted to establish the most critical features for the software.

Requirements were gathered in two main ways. First, an interview was conducted with a member of last year's team working on the project. This involved both a discussion of the current state of the software, and common problems encountered with the flight tests.

Requirements were also gathered by observation. This year's team conducted several flight tests to become familiar with the equipment in use. This provided an opportunity to see first hand problems with takeoff.

The notes from these two different requirements gathering exercises were then combined and developed into a draft list of user stories. A list of stories are given in the appendix. These stories were then prioritised based on discussions with the team and the project supervisors.

Day to day work on the project is allocated by dividing the user story into a number of tasks, and selecting a task to work on. Progress on tasks is discussed with supervisors and other team members at our regular meetings twice a week.

Source code is version controlled using Git and the University of Canterbury GitLab server (eng-git). System documentation and records of meetings are also kept here. Larger files, such as video recordings of flight tests and telemetry logs are stored in a university OneDrive cloud storage system.

V. Discussion / Evaluation

A. Transfer Time

The time to build and transfer software to the drones is highly variable, depending on how much of Docker's caching can be used. Building a docker image that can run the code produced by other sub-projects from scratch can take as long as 15 minutes. This is because all the dependencies, including the operating system need to be downloaded and installed. The process is further slowed by the fact that the images have to be built for the arm architecture of the Raspberry Pi, which requires emulation on the standard desktop x86-64 architecture.

This is less of a concern if we rely on the automatic building from the continuous integration, because it can build in the background and then be finished by the time we're ready to transfer the code on to the drones.

On the other hand, if updating an image from a previous version that already has the dependencies installed, the transfer can take place in approximately 30 seconds. In fact, the actual software transfer takes up a very small portion of this time. The majority of the time is spent on generating the certificates, transferring them over to the drones, and restarting Docker on the drones so that they will accept the new certificates. This means that as larger numbers of drones are added to the formation, the total transfer time is unlikely to significantly increase - most of the time is a fixed cost.

B. User Interface Experience

The user interface is clean and easily readable, allowing access to all functionality provided by the back-end. A screenshot is shown in Figure 4. You can download software from the virtual machine over the internet or build it locally, then transfer that software to the drones. Finally, you can launch the software on all the drones with a single command.

[5] See https://flask.palletsprojects.com/
[6] See https://docs.python.org/3/library/subprocess.html

The user interface does have some usability flaws. As noted in the project demonstration, there is limited feedback available to the user to show that an action has completed. For example, after transferring the software there is nothing to show that the software has successfully transferred. This makes it much harder to use.

This functionality could be added fairly easily by responding to the returned status code from the back-end, but was not completed as time ran short.

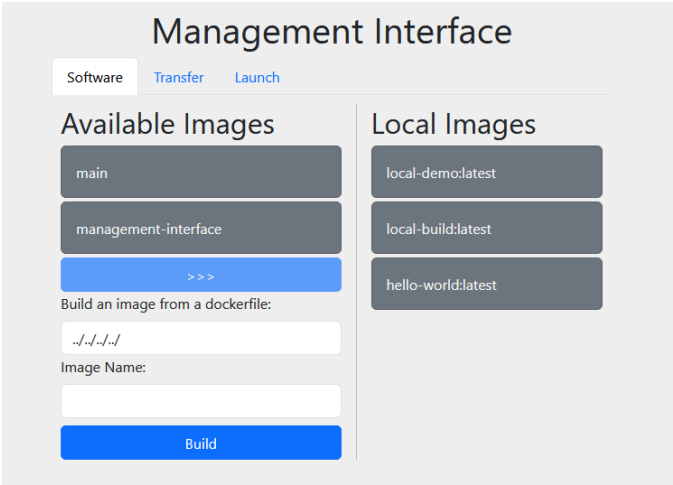

Fig. 4. The front-end of the management-interface.

C. Drone Configuration

One aspect of this software worth considering is that it does require more initial set up than the former plan. In order to get the drones set up for the software transfer, you have to configure and install docker on each drone. You also need to download and start up the small web server that allows the management interface to detect that the device is a drone. Finally, you need to transfer across keys that allow the management interface to connect using SSH to run commands and copy the security certificate.

This does mean it would take slightly longer to add a new drone to the formation, although this could be mitigated by simply having a base image containing docker and the web server that you can install directly on the drones.

VI. Conclusion

Drones in formation have a number of real-world applications. In this paper, we describe a solution that streamlines the deployment of software onto a number of drones. This deployment allows for faster iterations during development and ensures that the software is consistent across all drones.

The solution was a significant improvement over the manual system, although there are still improvements to be made.

A. Further Research

There are many minor improvements that could be made to the management interface. Better feedback for performing actions would greatly improve usability. Certain features like re-using a certificate if it is still valid could serve to speed up transfer time.

In future we would like to see this work better integrated with some of the work of other team members. As a part of their work on the project they record a lot of telemetry and examine it after a flight test to determine the success of the flight test. The telemetry and visualisations they developed could be included in the management interface so it could be displayed in real time.

Another area is that currently the formation's configuration files have to be manually written in JSON. If the management interface could automatically generate these configuration files, it would significantly speed up the other project's control loop tuning.

A more long term research goal would be to implement a system similar to that created by Varadharajan et al. [6], which would allow for a much more decentralised software transfer system for even greater flexibility, although with significantly more complexity.

References

[1] R. J. Francis, K. J. Brandis, and J. A. McCann, "Offline imagery checks for remote drone usage," *Drones*, vol. 6, no. 12, 2022, ISSN: 2504-446X. DOI: 10.3390/drones6120395.

[2] A. Lavrenko, Z. Barry, R. Norman, *et al.*, "Autonomous swarm of UAVs for tracking of flying insects with harmonic radar," in *2021 IEEE 93rd Vehicular Technology Conference (VTC2021-Spring)*, Journal Abbreviation: 2021 IEEE 93rd Vehicular Technology Conference (VTC2021-Spring), Apr. 25, 2021, pp. 1–5, ISBN: 2577-2465. DOI: 10.1109/VTC2021-Spring51267.2021.9449074.

[3] Juan Jesús Roldán-Gómez, E. González-Gironda, and A. Barrientos, "A survey on robotic technologies for forest firefighting: Applying drone swarms to improve firefighters' efficiency and safety," *Applied Sciences*, vol. 11, no. 1, p. 363, 2021, Place: Basel Publisher: MDPI AG. DOI: 10.3390/app11010363. [Online]. Available: https://www.proquest.com/scholarly-journals/survey-on-robotic-technologies-forest/docview/2559415075/se-2.

[4] S. M. Brecher, "Microprocessor firmware update inventory model," *The Bell System Technical Journal*, vol. 60, no. 10, pp. 2293–2306, Dec. 1981, Conference Name: The Bell System Technical Journal, ISSN: 0005-8580. DOI: 10.1002/j.1538-7305.1981.tb00227.x. [Online]. Available: https://ieeexplore.ieee.org/document/6772263 (visited on 10/26/2023).

[5] M. La Manna, L. Treccozzi, P. Perazzo, S. Saponara, and G. Dini, "Performance evaluation of attribute-based encryption in automotive embedded platform for secure software over-the-air update," *Sensors (Basel, Switzerland)*, vol. 21, no. 2, p. 515, 2021, ISBN: 1424-8220.

[6] V. S. Varadharajan, D. S. Onge, C. Guss, and G. Beltrame, "Over-the-air updates for robotic swarms," *IEEE software*, vol. 35, no. 2, pp. 44–50, 2018, ISBN: 0740-7459.

[7] "Docker: Accelerated container application development." (May 10, 2022), [Online]. Available: https://www.docker.com/ (visited on 10/26/2023).

[8] E. A. Santos, C. McLean, C. Solinas, and A. Hindle, "How does docker affect energy consumption? evaluating workloads in and out of docker containers," *Journal of Systems and Software*, vol. 146, pp. 14–25, Dec. 1, 2018, ISSN: 0164-1212. DOI: 10.1016/j.jss.2018.07.077. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121218301456 (visited on 10/26/2023).

[9] C. Anderson, "Docker [software engineering]," *IEEE Software*, vol. 32, no. 3, pp. 102–c3, May 2015, Conference Name: IEEE Software, ISSN: 1937-4194. DOI: 10.1109/MS.2015.62. [Online]. Available: https://ieeexplore.ieee.org/document/7093032 (visited on 10/26/2023).

Appendix

A. User Stories

U1: Overview display:

Create a screen that shows an overview of all drones. For each drone, it should display the name of the drone and a few pieces of critical information. For now, we will assume that to be the battery level.

U2: Remote reboot:

Many problems in field tests last year could be fixed by rebooting the drone. Functionality should exist to reboot both the Raspberry Pi and the PX4 remotely, rather than requiring the battery to be plugged and unplugged.

U3: Individual drone process statuses:

To diagnose the cause of issues, users should be able to see the status of various processes inside the drone (the result of systemctl status $process). Users should also be able to stop and restart relevant processes here.

U4: Automatically update drone software remotely:

A common problem was not knowing if a drone was running the most up to date version of the software. Changes to code and other files made on the gitlab should be able to be sent to drones with a single action. This includes configuration data for the px4.

U5: Individual drone ROS data:

For more detailed monitoring and debugging of code in the field. When a particular drone is clicked, users should be able to input the name of a ROS topic and watch all messages sent across this topic.

U6: Real time database access:

Sub-project 1 implements a real time database that is shared between the different drones in the formation. Users should be able to see the information stored in this database.

U7: Live telemetry:

The system should be able to receive data such as position and velocity from the PX4 and display it with minimal lag.

B. User Manual

1) Building: Before you start, you will need a working installation of the docker engine. Instructions can be found at https://docs.docker.com/engine/install/

a) Backend:

The backend of the interface is written as a flask webserver, using python.

First, make sure you have python3 installed

sudo apt update && sudo apt install python3

To install the dependencies, I recommend creating a virtual environment.

cd uav-formations python3 -m venv ./venv source venv/bin/activate

Then the dependencies can be installed to that virtual environment using

pip3 install -r src/2023/management_interface/ back-end/requirements.txt

You should then be able to launch the backend with:

sudo ./venv/bin/python3 src/2023/management-interface/backend/controller.py

Unfortunately sudo is required for the drone detection features.

b) Frontend:

The frontend is written using react and typescript. You will need to have npm installed.

Instructions for installing npm can be found here[7]

Once you have npm installed, cd into the directory and install the dependencies

cd frontend npm install

You can then launch the frontend with

npm run start

2) Usage: The management interface consists of three pages.

a) Software:

Before going out into the field, you should use this page to chose what software you would like to take with you so that you can transfer it on to the drones later. You can pick from pre-built images based on code that you have pushed to the eng-git server, or build an image from the current code on your local machine.

b) Transfer:

The transfer page shows the drones it has detected connected to the wifi network, and also lists the software you have available on your local machine. Select the drones you would like to transfer to, the image you wish to transfer, and click transfer software to start. The process might be quite slow if this is the first image that has been sent to a particular drone, but later images should be faster.

c) Launch:

When in the field, you can start the software running on particular drones by selecting them, inputting the config folder you would like them to use, and then hitting launch. The drones should soar gracefully into the air :)

[7] https://docs.npmjs.com/downloading-and-installing-node-js-and-npm

3) Details: There are two main parts of this project. The first is the virtual machine, which automatically builds and stores code pushed to the eng-git server.

a) Virtual Machine:

When inside the uc network, you can access the virtual machine over ssh (password: #######)

ssh user@csse-sdff.canterbury.ac.nz

When you make a push to a branch, eng git uses the .gitlab-ci.yml file to run continuous integration. It sends that job to the gitlab runner, which clones the repo, before running the commands listed under script:

These commands build the docker image, and then make it publicly available from https://csse-sdff.canterbury.ac.nz If you want to test out an image now, you can run

docker login csse-sdff.canterbury.ac.nz # Username: test, Password: ####### docker pull csse-sdff.canterbury.ac.nz/main #Or whatever branch name you would like the image for

Note that in the script we use a custom builder, and specify that when we build it we would like the platform to be `linux/arm64/v8`. If we do not do this, the software will be incompatible with the drones' processor.

b) Management-Interface:

Detecting Drones

We don't know ahead of time what the IPs of our drones will be if they are assigned using dhcp. For this reason, we have to automatically detect them. To do this, we first identify all devices connected to the wifi network by sending a packet to the broadcast MAC address. Then, we go through all the devices and ask if they are a drone by sending an HTTP request to {their_ip}:5000/isDrone. Actual drones should respond with a 200 status, whereas other devices will not.

Transferring Software

The second important part is being able to transfer the software in the field without internet access. The management interface allows you to pull down images and store them on your laptop, then transfer them to the drones later.

In order to transfer the images from the laptops to the drones, the system goes through a number of steps

1. On launch, create a certificate using `openssl` and start up the docker registry. We will use these later.
2. When a transfer is requested, copy the certificate over to the drone (using scp), and call `update ca-certificates` to get docker on the drones to accept the new certificate.
3. Push the image requested for transfer to the docker registry
4. Synchronise the time on the laptop with the time on the drones. We need this for the security certificate validation, but it is also handy for logging purposes.
5. Run `docker pull` on the drones to begin the transfer process.

(Redacted passwords are in the eng-git repository, https://eng-git.canterbury.ac.nz/eng42/uav-formations)